[NOMBRE DE LA INSTITUCIÓN]

[FACULTAD / PROGRAMA EDUCATIVO]

PROYECTO INTEGRADOR

Pipeline CI/CD automatizado para API REST con Docker, GitHub Actions y AWS EC2

Asignatura: [Nombre de la materia]

Docente: [Nombre del docente]

Integrantes: [Nombres completos]

Grupo: [Grupo]

[Ciudad, Estado]

Octubre de 2026

Índice

1. Introducción **1**

2. Objetivos **2**

2.1. Objetivo general 2

2.2. Objetivos específicos 2

3. Arquitectura y procedimiento **3**

4. Resultados **3**

5. Seguridad y limitaciones **4**

6. Conclusiones **5**

7. Fuentes de información **5**

1. Introducción

El desarrollo de software actual requiere entregar cambios con frecuencia sin perder control sobre la calidad, la seguridad ni la disponibilidad de los servicios. Cuando la compilación, las pruebas y la publicación se realizan manualmente, cada paso depende de que una persona recuerde ejecutar los comandos correctos y documente sus resultados. Esa dependencia hace más probable que una modificación que funciona en el equipo de desarrollo falle al ejecutarse en otro entorno. La integración continua y la entrega continua, conocidas como CI/CD, responden a este problema mediante procesos repetibles que verifican y distribuyen el software a partir de cambios versionados. En este proyecto se implementa un flujo automatizado para una API REST construida con Node.js y Express, cuya imagen se publica en Docker Hub y puede desplegarse en una instancia Ubuntu de Amazon EC2.

La integración continua comienza con una modificación enviada a la rama principal o propuesta mediante un pull request. GitHub Actions instala las dependencias desde el archivo de bloqueo y ejecuta pruebas automatizadas de integración sobre las rutas HTTP. El proyecto expone diez recursos de ejemplo: usuarios, productos, categorías, pedidos, clientes, proveedores, inventario, carritos, reseñas y pagos. Cada recurso ofrece operaciones para listar, crear, consultar, reemplazar, modificar parcialmente y eliminar registros. En conjunto son sesenta rutas CRUD, además de un endpoint de salud. Las pruebas comprueban tanto los casos exitosos como validaciones de entrada y respuestas para recursos inexistentes. Un umbral de cobertura global del setenta por ciento impide considerar aprobada una ejecución con una proporción menor de código cubierto.

La automatización no termina al probar el código. Una vez aprobadas las pruebas en un push a `main`, el pipeline construye una imagen Docker y publica en Docker Hub dos etiquetas: `latest` para la versión más reciente y el SHA completo del commit para identificar de manera inmutable el código asociado. El empaquetado en contenedores reduce diferencias entre la máquina de desarrollo y el servidor, pues agrupa el runtime de Node.js, las dependencias de producción y la aplicación en una unidad reproducible. El `Dockerfile` usa una imagen base ligera, omite las dependencias de desarrollo y ejecuta el proceso con un usuario sin privilegios de administrador. El archivo `.dockerignore` limita el contexto de construcción y excluye, entre otros elementos, dependencias locales, archivos de entorno y claves.

La entrega continua hacia la nube se realiza por SSH a una instancia EC2 con Ubuntu. GitHub Secrets almacena el usuario y token de Docker Hub, el host, el usuario de SSH, la clave privada y la huella de la clave de host; de ese modo, los valores confidenciales no se escriben en el código fuente. Nginx escucha en el puerto HTTP público y actúa como proxy inverso hacia uno de dos contenedores de la API, enlazados únicamente a la interfaz local. El despliegue descarga la imagen identificada por el commit, inicia el contenedor inactivo y consulta su endpoint de salud antes de cambiar el upstream de Nginx. Después de validar la nueva versión, recarga el proxy y retira el contenedor anterior. Este patrón blue-green reduce

la ventana de interrupción frente a detener primero la versión activa, aunque la disponibilidad final también depende de la salud de la instancia, la red y la configuración de AWS.

El objetivo del trabajo es demostrar un recorrido automatizado, verificable y documentado desde el código fuente hasta una API accesible por HTTP. Para alcanzarlo se preparan endpoints funcionales, pruebas de integración con reporte de cobertura, una imagen de contenedor y un workflow que separa la validación de la publicación y del despliegue. También se documentan las tareas que requieren intervención del equipo: crear el repositorio Docker Hub, aprovisionar la instancia, limitar los puertos mediante el Security Group, registrar secretos de GitHub y habilitar el despliegue. Estas tareas no pueden completarse de forma segura sin las credenciales y decisiones del propietario de la cuenta.

El alcance de la API es académico y deliberadamente explícito: los datos viven en memoria y se pierden cuando el proceso se reinicia, por lo que el servicio demuestra el ciclo CI/CD y las operaciones HTTP, pero no ofrece persistencia ni está destinado a datos reales. Asimismo, la disponibilidad pública de la URL depende de que el equipo cree la instancia y complete la configuración de AWS. El informe distingue los resultados medidos localmente de los resultados en nube que todavía deben obtenerse en la cuenta institucional. Esta separación evita presentar como ejecutados despliegues, capturas o verificaciones que requieren recursos externos.

2. Objetivos

2.1. Objetivo general

Implementar y validar un pipeline CI/CD que pruebe una API REST, publique su imagen versionada en Docker Hub y permita desplegarla de forma blue-green en AWS EC2.

2.2. Objetivos específicos

- Implementar al menos sesenta operaciones HTTP funcionales distribuidas en recursos CRUD.
- Automatizar pruebas de integración y exigir un mínimo global de 70 % de cobertura.
- Construir una imagen Docker reducida que no incluya dependencias de desarrollo ni secretos.
- Configurar GitHub Actions para probar cambios y publicar las etiquetas `latest` y SHA en Docker Hub.
- Documentar el aprovisionamiento de EC2, la protección de credenciales y el procedimiento de despliegue.

3. Arquitectura y procedimiento

La API escucha en el puerto 3000 dentro de su contenedor. En EC2, Nginx recibe las peticiones del puerto 80 y las reenvía al puerto local del contenedor activo (8081 o 8082). Los puertos de aplicación no se exponen directamente a Internet. El script de preparación instala Docker y Nginx, configura el proxy y deja establecido el slot inicial. En cada despliegue, Actions usa SSH con verificación de huella, descarga el tag del commit, levanta el slot alternativo, comprueba `/api/health`, actualiza la configuración y recarga Nginx.

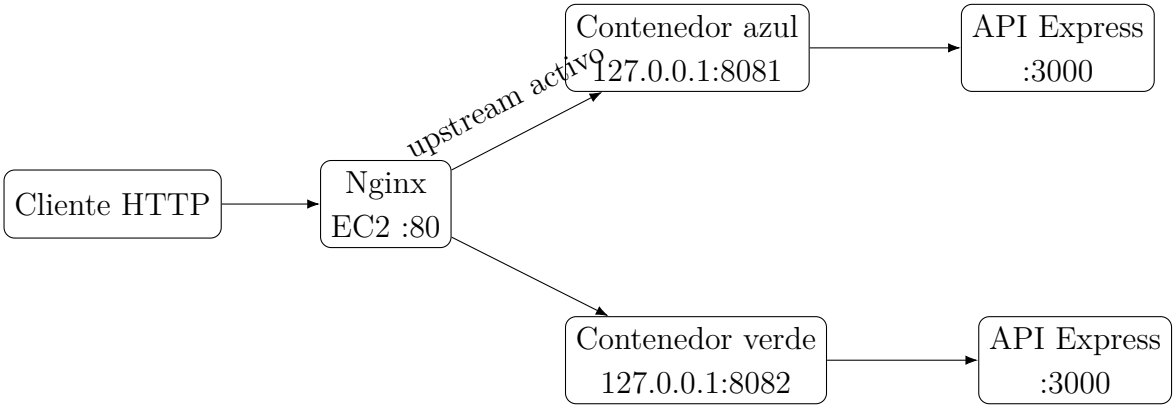


Figura 1: Arquitectura de ejecución y cambio blue-green en EC2.

La Figura 1 muestra que solo Nginx publica el puerto HTTP. El segundo contenedor se prepara en paralelo y recibe tráfico después de superar el health check.



Figura 2: Etapas del workflow y condición de publicación/despliegue.

La Figura 2 resume las dependencias: ninguna imagen se publica si fallan las pruebas. Los pull requests ejecutan validación; publicar requiere un push a `main`, y desplegar requiere además habilitar la variable `ENABLE_EC2_DEPLOY`.

4. Resultados

Las pruebas locales se ejecutaron con el runner nativo de Node.js y Supertest sobre los diez recursos CRUD y el endpoint de salud. Se aprobaron 19 casos de prueba. El informe

de cobertura agregado reportó 93.8 % de sentencias, 97.05 % de ramas y 93.8 % de líneas, por encima del mínimo del 70 %. El archivo de la aplicación que implementa las rutas alcanzó 100 % en estas métricas; la cobertura global incluye también el punto de entrada del servidor, que no se inicia en las pruebas de integración.

Indicador local	**Resultado**
Operaciones CRUD implementadas	60
Pruebas aprobadas	19 de 19
Cobertura de sentencias	93.8 %
Cobertura de ramas	97.05 %
Cobertura de líneas	93.8 %
Umbral configurado	70 %

Figura 3: Resultados medidos localmente con c8 y el runner de Node.js.

La Figura 3 presenta las mediciones locales obtenidas en esta versión. La construcción de la imagen y el despliegue real en Docker Hub y EC2 quedan pendientes hasta que el equipo configure su daemon Docker, sus credenciales y la instancia; no se atribuyen a este trabajo resultados externos que todavía no se han observado. Una vez configurados, deben anexarse capturas del workflow completado, los tags de la imagen y la respuesta HTTP pública.

5. Seguridad y limitaciones

Las credenciales se suministran mediante GitHub Secrets y no se guardan en los archivos del proyecto. La huella del host SSH se compara en la acción de despliegue para evitar confiar automáticamente en una clave recibida durante la conexión. El Security Group debe restringir el puerto 22 a las direcciones de administración y permitir el puerto 80 para clientes HTTP. La imagen ejecuta la aplicación como usuario no privilegiado y solo necesita exponer el puerto interno 3000.

La limitación funcional más importante es el almacenamiento en memoria, que no sobrevive a reinicios y no proporciona transacciones ni aislamiento persistente. La configuración de ejemplo sirve para demostrar el flujo y no reemplaza una arquitectura de producción con base de datos, HTTPS, monitoreo, política de copias de seguridad y gestión formal de secretos. El cambio blue-green minimiza la interrupción, pero no constituye por sí solo una garantía contractual de disponibilidad.

6. Conclusiones

Se implementó un flujo reproducible que integra una API REST, pruebas automatizadas, cobertura mínima obligatoria, contenedorización y publicación de versiones identificables. La ejecución local verificó las sesenta operaciones CRUD y alcanzó coberturas superiores al 90 % en las cuatro métricas reportadas, por encima del umbral establecido. La identificación de las imágenes por SHA permite relacionar cada despliegue con el cambio exacto del repositorio y facilita reproducir una versión.

La automatización de nube se diseñó con separación entre preparación y despliegue: el equipo conserva sus credenciales en GitHub Secrets, limita la exposición de puertos y valida la nueva instancia de la API antes de cambiar el tráfico. Para completar la demostración solicitada todavía se requiere crear Docker Hub y EC2, registrar sus valores privados y ejecutar un push a `main`; solo después podrán documentarse una URL pública y capturas reales del pipeline. Por último, la persistencia en memoria delimita el propósito académico del prototipo y debe sustituirse por un almacenamiento durable antes de usarlo como servicio real.

7. Fuentes de información

1. GitHub Docs. “Understanding GitHub Actions”. https://docs.github.com/en/actions/about-github-actions. Consulta: 7 de octubre de 2026.

2. Docker Docs. “Dockerfile reference”. https://docs.docker.com/reference/dockerfile/. Consulta: 7 de octubre de 2026.

3. Docker Docs. “Build and push Docker images”. https://docs.docker.com/build/ci/github-actions/. Consulta: 7 de octubre de 2026.

4. Amazon Web Services. “Amazon EC2 User Guide”. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html. Consulta: 7 de octubre de 2026.

5. Express. “Express routing”. https://expressjs.com/en/guide/routing.html. Consulta: 7 de octubre de 2026.